

AI-Powered Face Recognition Attendance System

Complete Project Documentation

Version 1.0

Developed by **Lokeshwar Menati**
Django + OpenCV + Deep Learning Face Recognition

Generated: August 2026

Table of Contents

1. Executive Summary
2. Project Overview
3. Key Features
4. Technology Stack
5. System Architecture
6. Database Design
7. Face Recognition Engine
8. Application Modules & URL Routes
9. User Workflows
10. Installation & Setup
11. Usage Guide
12. Admin & Security
13. Project Directory Structure
14. API Reference
15. Limitations & Future Enhancements

1. Executive Summary

The **AI-Powered Face Recognition Attendance System** is a web-based application designed to automate student attendance tracking using facial recognition technology. Built with Django (Python) on the backend and OpenCV for computer vision, the system eliminates manual roll-call processes by identifying registered students through a webcam or browser camera feed in real time.

Students register by submitting personal details and a facial photograph. After administrator authorization, their face embeddings are used for recognition during attendance sessions. The system automatically records check-in and check-out times, prevents duplicate entries on the same day, and provides searchable attendance reports with duration calculations.

2. Project Overview

Project Name: AI-Powered Face Recognition Attendance System

Version: 1.0

Framework: Django 5.x

Primary Use Case: Educational institutions, training centers, and organizations requiring automated biometric attendance.

Repository: github.com/LokeshwarMenati/AI-Powered-Face-Recognition-Attendance-System

Problem Statement

Traditional attendance methods (paper registers, manual sign-in sheets) are time-consuming, prone to proxy attendance, and difficult to audit. This project addresses these challenges by providing a contactless, automated solution that leverages AI-based face recognition.

Solution

A Django web application that captures student face data during registration, stores face encodings derived from ONNX deep learning models (YuNet + SFace) or optionally FaceNet (PyTorch), and matches live camera frames against the authorized student database to mark attendance automatically.

3. Key Features

Feature	Description
Student Management	Register students with name, email, phone, class, and profile photo via webcam capture.
Authorization Workflow	Admin must authorize students before they participate in face recognition.
Real-Time Recognition	Browser-based webcam feeds frames to the server for face detection and matching.
Automatic Check-In/Out	First recognition marks check-in; second recognition (after 60 seconds) marks check-out.
Duplicate Prevention	One attendance record per student per day; prevents repeated check-ins.
Attendance Reports	Searchable list filtered by student name and date with duration calculation.
Camera Configuration	Configurable recognition threshold and camera source settings for admins.
Dashboard	Home page displays total students, attendance records, check-ins, check-outs, and cameras.

Feature	Description
Admin Panel	Django admin interface for managing students, attendance, and camera configs.
Dual Recognition Backend	Supports OpenCV YuNet/SFace ONNX models or PyTorch FaceNet as fallback.

4. Technology Stack

Component	Technology
Backend Framework	Django 5.0+ (Python web framework)
Database	SQLite (default, file-based)
Computer Vision	OpenCV (opencv-python-headless)
Face Detection	YuNet ONNX model (face_detection_yunet_2023mar.onnx)
Face Recognition	SFace ONNX model (face_recognition_sface_2021dec.onnx)
Optional ML Stack	PyTorch + facenet-pytorch (InceptionResnetV1 + MTCNN)
Image Processing	NumPy, Pillow (PIL)
Frontend	HTML5, CSS3, Bootstrap 5, Font Awesome, JavaScript
Authentication	Django built-in auth (session-based login/logout)
Media Storage	Local filesystem (MEDIA_ROOT/students/)
Deployment	Vercel configuration available (vercel.json)

5. System Architecture

The application follows a classic Django Model-View-Template (MVT) architecture with a browser-client capturing webcam frames and a server-side face recognition pipeline.

Architecture Layers

Layer	Responsibility
Presentation Layer	Django HTML templates with Bootstrap styling; JavaScript captures webcam frames as Base64 and sends to server.
Application Layer	Django views handle HTTP requests, authentication, CRUD operations, and orchestrate the recognition pipeline.
Business Logic Layer	Face detection, encoding, distance-based matching, attendance rules (check-in/out timing).
Data Layer	Django ORM models (Student, Attendance, CameraConfiguration) backed by SQLite.
ML/AI Layer	OpenCV FaceDetectorYN + FaceRecognizerSF (ONNX) or PyTorch FaceNet pipeline.

Data Flow (Attendance Marking)

1. User opens the Mark Attendance page and starts the browser webcam.
2. JavaScript periodically captures video frames and encodes them as Base64 JPEG.
3. Frames are POSTed to /api/recognize-face/ (recognize_face_api view).
4. Server decodes the image and runs face detection (YuNet or MTCNN).
5. Detected faces are aligned and converted to 128/512-dimensional feature vectors (SFace or FaceNet).
6. Feature vectors are compared against all authorized students' stored encodings using L2 distance.
7. If distance is below the configured threshold, the student is recognized.
8. Attendance logic: create record + check-in, or check-out if already checked in (after 60s cooldown).
9. JSON response returns recognized names, bounding boxes, and status messages to the browser.

6. Database Design

6.1 Student Model

Field	Type	Description
name	CharField(255)	Full name of the student
email	EmailField(255)	Contact email address
phone_number	CharField(15)	Phone number
student_class	CharField(100)	Class or batch identifier
image	ImageField	Profile photo stored in media/students/
authorized	BooleanField	Default False; must be True for recognition

6.2 Attendance Model

Field	Type	Description
student	ForeignKey(Student)	Reference to the student
date	DateField	Auto-set to current date on creation
check_in_time	DateTimeField	Timestamp when student was first recognized
check_out_time	DateTimeField	Timestamp when student checked out

Methods: `mark_checked_in()`, `mark_checked_out()`, `calculate_duration()` — returns formatted duration string (e.g., '2h 30m 15s').

6.3 CameraConfiguration Model

Field	Type	Description
name	CharField(100, unique)	Descriptive name for the camera setup
camera_source	CharField(255)	Camera index (0) or RTSP/HTTP URL for IP cameras
threshold	FloatField	Recognition confidence threshold (default 0.6)

7. Face Recognition Engine

The system implements a dual-backend approach for maximum compatibility across environments.

7.1 Primary: OpenCV YuNet + SFace (ONNX)

When ONNX model files are present in the `models/` directory, the system uses:

- **YuNet** (`face_detection_yunet_2023mar.onnx`) — fast, lightweight face detection
- **SFace** (`face_recognition_sfce_2021dec.onnx`) — face alignment and 128-D feature extraction

This approach requires no GPU and works with opencv-python-headless, making it ideal for lightweight deployments.

7.2 Fallback: PyTorch FaceNet

If PyTorch and facenet-pytorch are installed, the system uses MTCNN for face detection and InceptionResnetV1 (pretrained on VGGFace2) for 512-dimensional embeddings. This provides higher accuracy but requires more resources.

7.3 Matching Algorithm

For each detected face, the system computes L2 (Euclidean) distance between the test encoding and all known encodings. The closest match below the threshold (default 0.6, configurable via CameraConfiguration) is accepted. Otherwise, the face is labeled 'Not Recognized'.

Key Functions (views.py):

- `detect_and_encode(image)` — detects faces and returns encodings + bounding boxes
- `encode_uploaded_images()` — loads authorized student photos and generates known encodings
- `recognize_faces(known_encodings, known_names, test_encodings, threshold)` — performs matching

8. Application Modules & URL Routes

URL	View Name	Description	Access
/	home	Dashboard with statistics	Public
/capture_student/	capture_student	Student self-registration with webcam	Public
/selfie-success/	selfie_success	Registration confirmation page	Public
/capture-and-recognize/	capture_and_recognize	Mark attendance via webcam	Public
/api/recognize-face/	recognize_face_api	JSON API for face recognition	Public (CSRF exempt)
/students/attendance/	student_attendance_list	Attendance reports with search/filter	Public
/students/	student_list	List all students	Admin only
/students/<pk>/	student_detail	View student details	Admin only
/students/<pk>/authorize/	student_authorize	Authorize/deauthorize student	Admin only
/students/<pk>/delete/	student_delete	Delete student record	Admin only
/login/	user_login	Admin login page	Public
/logout/	user_logout	Logout and redirect to login	Authenticated
/camera-config/	camera_config_create	Create camera configuration	Admin only
/camera-config/list/	camera_config_list	List camera configurations	Admin only
/camera-config/update/<pk>/	camera_config_update	Edit camera configuration	Admin only
/camera-config/delete/<pk>/	camera_config_delete	Delete camera configuration	Admin only
/admin/	Django Admin	Built-in admin panel	Superuser

9. User Workflows

9.1 Student Registration Workflow

1. Navigate to Student Registration (/capture_student/).
2. Fill in name, email, phone number, and class.
3. Capture a clear face photo using the browser webcam.
4. Submit the form — student is saved with authorized=False.
5. Admin reviews and authorizes the student from the Manage Students section.

9.2 Attendance Marking Workflow

1. Ensure at least one student is authorized with a clear profile photo.
2. Navigate to Mark Attendance (/capture-and-recognize/).
3. Click Start Camera and allow browser camera permissions.

4. Stand in front of the camera — the system auto-detects and recognizes faces.
5. First recognition: Check-in recorded with timestamp.
6. Second recognition (after 60 seconds): Check-out recorded.
7. View results in Attendance Details with date and duration.

9.3 Admin Workflow

1. Create superuser: `python manage.py createsuperuser`
2. Login at `/login/` with admin credentials.
3. Review registered students and authorize eligible ones.
4. Configure camera settings (threshold, source) if needed.
5. Monitor attendance reports and manage records via admin panel.

10. Installation & Setup

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)
- Webcam or camera device (for browser-based capture)
- Optional: CMake and Visual Studio C++ Build Tools (if using dlib/face_recognition)

Installation Steps

```
git clone
https://github.com/LokeshwarMenati/AI-Powered-Face-Recognition-Attendance-System.git

cd
AI-Powered-Face-Recognition-Attendance-System/Project-Face-attendance-system-version-1.0

python -m venv venv

venv\Scripts\activate (Windows)
source venv/bin/activate (macOS/Linux)

pip install -r requirements.txt

python manage.py migrate

python manage.py createsuperuser (optional)

python manage.py runserver
```

Dependencies (requirements.txt)

Package	Version	Purpose
Django	>=5.0.0	Web framework
numpy	latest	Numerical computing for face vectors
opencv-python-headless	latest	Computer vision without GUI dependencies
Pillow	latest	Image processing and ImageField support

ONNX Model Files

Place the following models in the models/ directory:

- face_detection_yunet_2023mar.onnx — YuNet face detector
- face_recognition_sface_2021dec.onnx — SFace recognizer

These can be downloaded from the OpenCV Zoo repository.

11. Usage Guide

After starting the server, open **http://127.0.0.1:8000/** in a modern web browser (Chrome or Firefox recommended for webcam access).

Page	How to Use
Dashboard	View total students, attendance records, check-ins, check-outs, and configured cameras.
Student Registration	Public page for new students to register with webcam photo capture.
Manage Students	Admin-only: view, authorize, and delete student records.
Mark Attendance	Open webcam interface; faces are recognized and attendance is auto-recorded.
Attendance Details	Search by student name, filter by date, view check-in/out times and duration.
Manage Cameras	Admin-only: create/edit/delete camera configurations and recognition thresholds.

12. Admin & Security

Authentication

Django session-based authentication protects admin-only views. The `@login_required` and `@user_passes_test(is_admin)` decorators restrict student management and camera configuration to superusers. Public pages (registration, attendance marking, reports) are accessible without login.

Django Admin Panel

Available at `/admin/`. Registered models: Student (with filters by class and authorization), Attendance (with date filters; check-in/out times are read-only on edit), and CameraConfiguration.

Security Notes for Production

- Change `SECRET_KEY` in `settings.py` before deployment.
- Set `DEBUG = False` in production.
- Configure `ALLOWED_HOSTS` appropriately (currently set to `'*'`).
- Use HTTPS for webcam data transmission.
- Consider adding CSRF protection to the `recognize_face_api` endpoint.
- Migrate from SQLite to PostgreSQL for production workloads.
- Implement rate limiting on the recognition API.

13. Project Directory Structure

Project-Face-attendance-system-version-1.0/	
■■■ manage.py	# Django management script
■■■ requirements.txt	# Python dependencies
■■■ db.sqlite3	# SQLite database (after migrate)
■■■ Project101/	# Django project settings
■ ■■■ settings.py	# Configuration (DB, media, timezone)
■ ■■■ urls.py	# Root URL routing
■ ■■■ wsgi.py / asgi.py	# Server entry points
■■■ app1/	# Main application
■ ■■■ models.py	# Student, Attendance, CameraConfiguration
■ ■■■ views.py	# All view logic + face recognition
■ ■■■ urls.py	# App URL patterns
■ ■■■ forms.py	# Django forms
■ ■■■ admin.py	# Admin panel configuration
■ ■■■ migrations/	# Database migration files
■■■ templates/	# HTML templates
■ ■■■ home.html	# Dashboard
■ ■■■ capture_student.html	# Registration form
■ ■■■ capture_and_recognize.html	# Attendance webcam UI
■ ■■■ student_list.html	# Student management
■ ■■■ student_attendance_list.html	# Reports
■ ■■■ ...	# Other templates
■■■ models/	# ONNX model files
■ ■■■ face_recognition_sface_2021dec.onnx	
■■■ media/students/	# Uploaded student photos
■■■ staticfiles/	# Collected static files

14. API Reference

POST /api/recognize-face/

Description: Accepts a Base64-encoded JPEG image from the browser webcam and returns face recognition results with attendance actions.

Request: POST with form data

- image_data: Base64 string (data:image/jpeg;base64,...)

Response (JSON):

- recognized: boolean — whether any face was matched
- faces: array of {name, box, recognized}

- messages: array of attendance status strings
- error: string (on failure)

Attendance Rules:

- New record for today → mark_checked_in()
- Existing record, no check-out, ≥ 60 seconds since check-in → mark_checked_out()
- Already checked in (< 60 s) → 'already checked in' message
- Already checked out → 'already checked out' message

15. Limitations & Future Enhancements

Current Limitations

- Recognition accuracy depends on lighting, angle, and photo quality.
- Only authorized students with stored photos can be recognized.
- Single-day attendance model (one check-in/out pair per day).
- SQLite may not scale for large institutions.
- No email/SMS notifications for attendance events.
- YuNet detection model file may need separate download.

Suggested Future Enhancements

- Multi-face attendance in a single frame with batch processing.
- Export attendance reports to CSV/PDF/Excel.
- Role-based access (teachers vs. administrators).
- Integration with IP cameras via RTSP streams.
- Liveness detection to prevent photo spoofing.
- Mobile app or PWA for remote attendance.
- PostgreSQL/MySQL support and cloud deployment.
- Real-time dashboard with charts and analytics.